

EVALUATION & EVALS

Evals are the difference between demo and deployment. Define success first.

DIMENSION	QUESTION
Outcome	Did the task complete? Is the answer right?
Process	Did it use the intended tools and steps?
Style	Did it follow the conventions you care about?
Efficiency	Did it avoid thrashing, waste, and retries?



RELIABILITY IMPROVEMENT ORDER

Do the cheap, high-signal fixes first.

#	STEP	SIGNAL
1	Clarify the task and success criteria	Highest
2	Add examples, constraints, output contracts	High
3	Ground with search, retrieval, or APIs	Medium
4	Choose the right model/reasoning level	Medium
5	Evaluate and iterate	Ongoing
6	Fine-tune after all above are in place	Late stage

Most failures are caused by underspecification or missing grounding, not by lack of tuning.

DATA & PRIVACY BOUNDARIES

APPROVED/CONTROLLED	EXTERNAL/THIRD PARTY
- Classify data before you paste it - Minimize what you send - Know what data is sent to which provider, what is retained, and what is used for training - Prefer approved organization environments	- Connectors have their own retention - Search and file tools can move data across boundaries - Use allowlists and explicit approval - Key question: what data reaches which model?

ANTI-PATTERNS: HOW LLM PROJECTS FAIL

ANTI-PATTERN	ROOT CAUSE
Vague prompt, blind trust	Task was underspecified
Stuff entire wiki into context	No curation of what matters now
Agent for a deterministic task	System was over-empowered
Broad credentials to save time	Permissions too wide
Measure vibes, not accuracy	Skipped the feedback loop
Skip transcript review	No error analysis

The fix is usually simple: smaller scope, better context, tighter permissions, explicit evals.

SECURITY RISKS (OWASP/NIST)

The biggest risks are around the model, not inside it.

THREAT	DESCRIPTION
Prompt Injection	Instructions and data share the same medium. Malicious text in documents, emails, or tool results can steer behavior.
Sensitive Info Disclosure	Model may leak training data, system prompts, or user-provided context to unintended recipients.
Improper Output Handling	Downstream systems or humans may execute text, code, or recommendations without adequate validation.
Excessive Agency	Too many tools, too much autonomy, too few guardrails.
System Prompt Leakage	Adversary extracts or infers system-level instructions, exposing policy, tools, or business logic.

Default Mitigations

- Separate instruction from data where possible
- Validate tool inputs and outputs
- Keep permissions narrow
- Require confirmation for consequential actions
- Log and review agent transcripts
- RAG and fine-tuning do not fully mitigate injection

A USEFUL PROMPT STRUCTURE

Most strong prompts include these layers. Use the minimum structure needed for the task.

#	LAYER	PURPOSE
1	Role/Persona	Set expertise and perspective
2	Task + Success	What to do and what “done” looks like
3	Context	Background data, constraints, environment
4	Constraints	Boundaries, edge cases, what to do when unsure
5	Output Format	Structure, length, schema, examples

- You are a staff security engineer.
- Write a design review for this AI workflow.
- Context: it sends meeting notes to a hosted LLM for summarization.
- Constraints:
 - identify privacy + injection risks
 - ask for clarification if data flow unclear
 - end with 3 concrete recommendations
- Output: markdown with headings + risk table

SOURCES & FURTHER READING

- OpenAI – A Practical Guide to Building Agents
- Anthropic – Building Effective AI Agents
- Anthropic – Effective Context Engineering for AI Agents
- Google – Prompt Design Strategies
- OWASP – Top 10 for LLM Applications
- NIST AI 600-1 – AI RMF: Generative AI Profile
- Model Context Protocol – Specification

SANS

OFFENSIVE OPERATIONS

LLM Usage Cheat Sheet v1.0

Author: Jean-François Maes

RELATED COURSES:

SEC565: Red Team Operations and Adversary Emulation

SEC699: Advanced Purple Teaming – Adversary Emulation & Detection Engineering

SANS.ORG/OFFENSIVE-OPERATIONS

PROMPTING TECHNIQUES

Few-Shot Examples

Show 1–3 examples of the desired output. Fastest way to transfer tone, formatting, and implicit style rules that are painful to explain in prose.

Decomposition/Deliberate Reasoning

For hard tasks, ask the model to break the task into steps, verify assumptions, or produce a concise rationale before the final answer. Prefer structured intermediate outputs over free-form reasoning.

Structured Output/Schema

When output must be parsed by code, provide a JSON schema or explicit format spec. Eliminates regex and post-processing.

Tool-Assisted Reasoning Loops

For tasks that require evidence or actions, let the model search, calculate, or call tools, then verify results before answering.

Negative Examples

Show what NOT to produce. Effective for style correction and avoiding common failure modes.

System Instructions

Higher-priority instructions supplied by the application. Typically persist within the current session or workflow. Use for role, tone, safety constraints, output contracts.

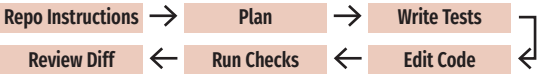
THE ABSTRACTION LADDER

Use the smallest effective abstraction. Do not jump to “agent” if a prompt or workflow will do.

LEVEL	WHAT	WHERE
1. Ask	Single answer, human judges	Default
2. Structured Prompt	Role + task + format + examples	Default
3. Grounded Workflow	Search, docs, or context cache	Often useful
4. Tool Assistant	Reads, calculates, calls APIs	When needed
5. Agent	Loops until an exit condition	Only when path is uncertain

Prefer workflows when the path is known; prefer agents when the path is uncertain and tool use must adapt dynamically.

CODING ASSISTANCE: SAFE DEFAULTS



AGENT.md / CLAUDE.md

- TypeScript with strict mode
- Use pnpm, not npm
- Never modify src/generated/
- Run tests before finalizing
- Max 200 lines per file
- Grant the least permissions needed
- Treat tests, linters, type checks as part of the spec
- Review the diff, not just the final explanation
- Prefer small, atomic commits
- Use a fresh context window for review (not the same session that wrote the code)

THE ABSTRACTION LADDER

Use LLMs as systems – with context, tools, policies, and evals – not as oracles. The human role is shifting upward: better specification, better verification, better system design.

AGENT WORKFLOW PATTERNS

The Core Loop

Agent performance depends on the loop, permissions, and verification layers around the model, not the model alone.



- Maximize a single agent first; split into multiple agents only when instructions or tools become too complex for one
- Each phase should ideally get a fresh context to avoid degradation
- Use checkpoints, approvals, and rollback paths for high-impact actions
- Prefer workflows when the path is known; agents when it is not

CONTROL	PURPOSE
State	Resumable memory across sessions
Checkpoints	Inspect before acting
Rollback	Undo bad steps
Approvals	Humans for high-impact decisions

Spec-Driven Development

For coding tasks, separate planning, execution, and review into distinct phases with structured handoffs. This prevents context degradation and keeps each phase focused.



TOOL USE & FUNCTION CALLING

- The model may choose tools and draft arguments, but the application should validate, constrain, and log consequential actions.
- Document tools like good APIs: clear name, description, parameter types
 - Prefer high-level tools over low-level primitives
 - Validate consequential calls before execution
 - Ask clarifying questions before risky or irreversible actions

MCP (Model Context Protocol)

An open protocol for connecting LLM applications to external tools and context sources in a standardized way. Think “USB-C for AI.” Defines Resources (data), Tools (actions), and Prompts (templates).

WHAT LLMS ARE GOOD AT VS. WEAK AT

EXCELLENT AT	WEAK UNLESS DESIGNED
• Summarizing, transforming tone and structure • Drafting, brainstorming, comparing options • Extracting patterns from messy language • Planning next steps when tools are visible • Classification and labeling	• Fresh facts without search or retrieval • Reliable structured output without schema • Irreversible actions without validation • Long contexts if you just stuff everything in • Security boundaries (attack surface is around the model)

GROUNDING: SOURCE OF TRUTH

Grounding tethers model output to verifiable sources. Pick the right one:

SOURCE	BEST FOR
Long Context	Corpus already fits and task is bounded; pair with caching when reused
File Search/RAG	Large or changing corpora, when citations matter
Web Search	Freshness, current events, when the answer may have changed
First-Party API	Authoritative business data, taking actions (tickets, DB, calendar)

ROLLOUT: PRODUCT, NOT NOVELTY

#	PHASE	ACTION
1	Pick a workflow	High-volume, clear ROI, bounded risk
2	Add controls	Grounding, permissions, approvals, logging
3	Instrument it	Track accuracy, cycle time, cost, failures
4	Expand carefully	Only after evals, reviews, and rollback paths

Operating Template

Owner/Use case/Allowed data/Grounding source/Enabled tools/Approval gates/Eval set/Rollback path

CONTEXT ENGINEERING

The question is not “how big is the window?” but “what belongs in it right now?”



- Curate the working set. Do not stuff everything in.
- Summarize stale detail. Restate critical constraints before next action.
- Long context does not equal good context.
- Use prompt chaining and compression for extended sessions.

Context Degradation WATCH OUT

As irrelevant, stale, or redundant context accumulates, model performance can degrade. Curate the working set, summarize stale details, and refresh context between task phases.

SAME TASK, FOUR LEVELS

Adding “/health endpoint” at each abstraction level:

LEVEL	WHAT IMPROVES	HUMAN ROLE
Chat	Speed only	Interpret
Structured	Specification	Specify
Grounded	Truth + relevance	Choose sources
Agent + verify	Execution + verification	Review + approve

KEY PROTOCOLS & TOOLS

NAME	FOCUS
MCP	Open protocol for standardized tool/data integration
CrewAI	Multi-agent orchestration with role-based agents
LangChain	Chains, agents, retrievers, tool integration
LiteLLM	Unified API across 100+ LLM providers

QUICK REFERENCE: CHECKLIST

DO

- Specify the task, audience, constraints, definition of done
- Ground important answers in search, retrieval, or APIs
- Keep permissions narrow and approvals explicit
- Read transcripts and diffs, not just final summaries
- Evaluate before you automate; automate before you scale
- Use fresh context windows for each phase of work
- Start with the most capable model, optimize down

AVOID

- Treating the model as an oracle for fresh facts
- Assuming long context = good context
- Giving broad credentials to “make the agent useful”
- Shipping systems you cannot measure or inspect
- Confusing more autonomy with more maturity
- Reviewing the code in the same session that wrote it